

Dicionários

3.1 O que é um dicionário?

Um dicionário armazena pares chave: valor. Em vez de acessar dados por posição numérica, você acessa por uma chave significativa. É a estrutura de dados mais importante do Python depois das listas.

□ Metáfora: O dicionário real

Num dicionário de português você não busca a palavra pela posição ('palavra número 4721').

Você busca pelo nome da palavra (chave) e encontra o significado (valor).

```
{'python': 'linguagem de programação de alto nível',  
'lista': 'estrutura de dados ordenada e mutável'}
```

3.2 Criação e acesso

```
# Criando um dicionário  
aluno = {  
    'nome':      'Carlos',  
    'idade':     17,  
    'curso':     'Desenvolvedor Python',  
    'nota':      8.5,  
    'aprovado':  True  
}  
  
# Acessando valores  
print(aluno['nome'])      # Carlos  
print(aluno['nota'])      # 8.5  
  
# Forma segura com .get() – sem KeyError se não existir  
print(aluno.get('email')) # None  
print(aluno.get('email', 'N/A')) # N/A (valor padrão)  
  
# Adicionando e modificando  
aluno['email'] = 'carlos@senai.df' # nova chave  
aluno['nota'] = 9.0                # atualiza valor existente  
  
# Removendo  
del aluno['aprovado']             # remove a chave  
nota = aluno.pop('nota')         # remove e retorna  
print(nota)                      # 9.0
```

3.3 Métodos essenciais

```
produto = {'nome': 'Notebook', 'preco': 3500.0, 'estoque': 12}

# Iterando
for chave in produto.keys():
    print(chave)

for valor in produto.values():
    print(valor)

for chave, valor in produto.items():
    print(f'{chave}: {valor}')

# Verificando existência de chave
print('preco' in produto)      # True
print('desconto' in produto)   # False

# Tamanho
print(len(produto))           # 3

# Copiando (nunca use dict2 = dict1 – copia a referência!)
copia = produto.copy()
```

3.4 Dicionários aninhados

Dicionários podem conter outros dicionários como valores, permitindo representar dados hierárquicos.

```
turma = {
    'Ana': {'nota': 9.0, 'frequencia': 90},
    'Bruno': {'nota': 6.5, 'frequencia': 80},
    'Carlos': {'nota': 4.0, 'frequencia': 60},
}

# Acessando dados aninhados
print(turma['Ana']['nota'])      # 9.0
print(turma['Carlos']['frequencia']) # 60

# Percorrendo a turma
for nome, dados in turma.items():
    nota = dados['nota']
    freq = dados['frequencia']
    situacao = 'Aprovado' if nota >= 7.0 and freq >= 75 else 'Reprovado'
    print(f'{nome}: {nota} | {freq}% | {situacao}')
```

3.5 Lista de dicionários — padrão muito comum

Na prática, você vai trabalhar frequentemente com listas de dicionários — o equivalente Python a uma tabela de banco de dados.

```
produtos = [
    {'nome': 'Notebook', 'preco': 3500.0, 'categoria': 'eletronico'},
    {'nome': 'Mouse', 'preco': 89.9, 'categoria': 'eletronico'},
    {'nome': 'Cadeira', 'preco': 650.0, 'categoria': 'mobilia'},
    {'nome': 'Monitor', 'preco': 1200.0, 'categoria': 'eletronico'},
]

# Filtrar eletrônicos
eletronicos = [p for p in produtos if p['categoria'] == 'eletronico']
print(len(eletronicos))    # 3

# Ordenar por preço
em_ordem = sorted(produtos, key=lambda p: p['preco'])
for p in em_ordem:
    print(f"{p['nome']:15} R$ {p['preco']:02f}")

# Total em estoque
total = sum(p['preco'] for p in produtos)
print(f'Total: R$ {total:02f}')
```